

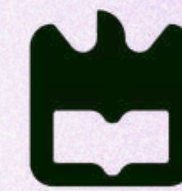
FINAL DELIVERABLE

nopCommerce

ARCHITECTURAL EVOLUTION

Presented by:

- Tomás Brás N° 112665
- Hugo Ribeiro N°113402
- Afonso Ferreira N°113480
- Rodrigo Abreu N°113626



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática



Computers

Electronics

Apparel

Digital downloads

Books

Jewelry

Gift Cards

Welcome to our store

Online shopping is the process consumers go through to purchase products or services over the Internet. You can edit this in the admin site.

If you have questions, see the [Documentation](#), or post in the [Forums](#) at [nopCommerce.com](#)

Electronics



Apparel



Digital downloads



Featured products



TABLE OF CONTENTS



**SCENARIO & FINAL
SCOPE**



**ARCHITECTURAL
EVOLUTION**



**QUALITY ATTRIBUTES
& EVIDENCE**



FINAL ADRS



LIVE DEMO



**DEGRADATION
& RECOVERY**



**TRADEOFFS
& LIMITATIONS**

SCENARIO & FINAL SCOPE

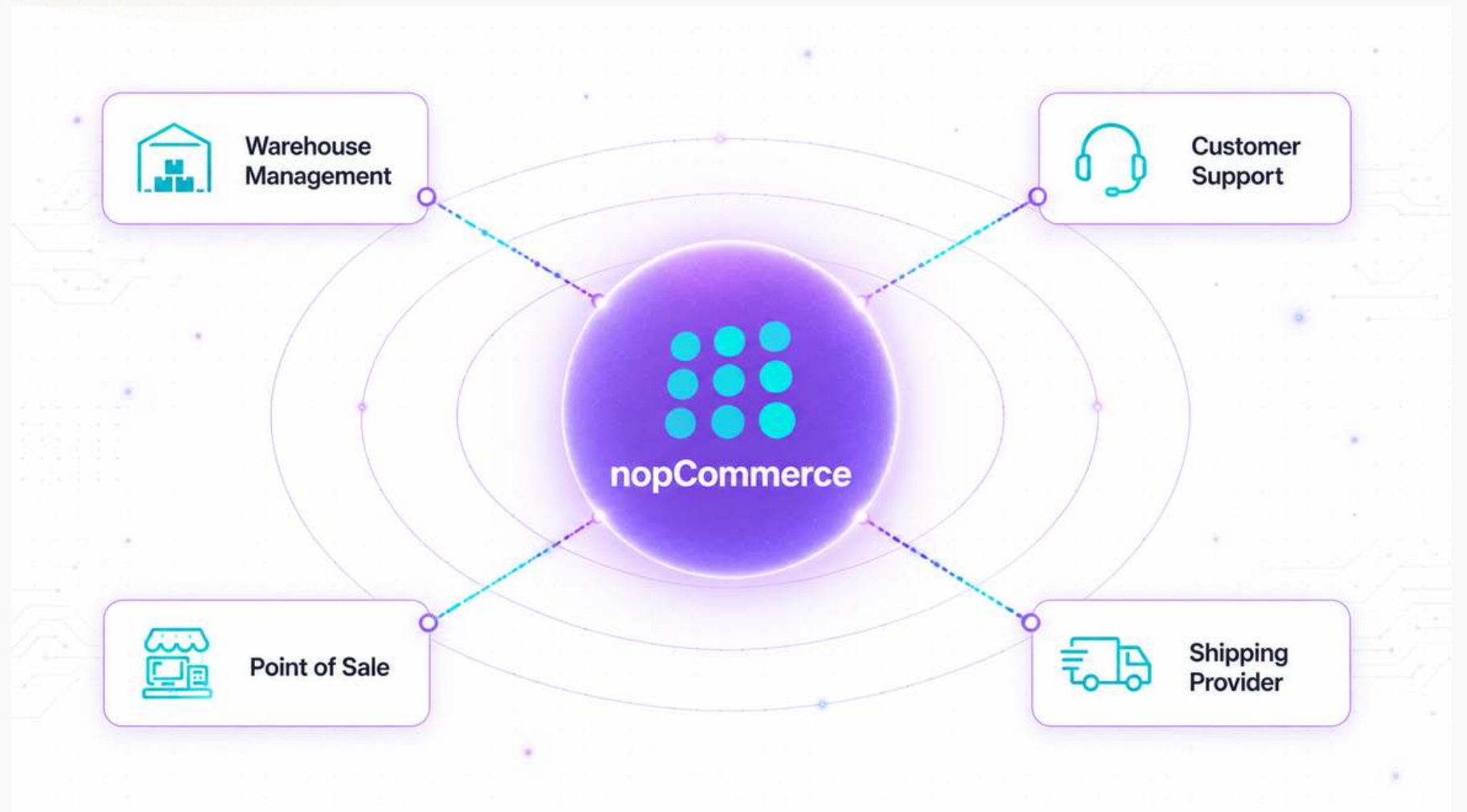
SCENARIO C

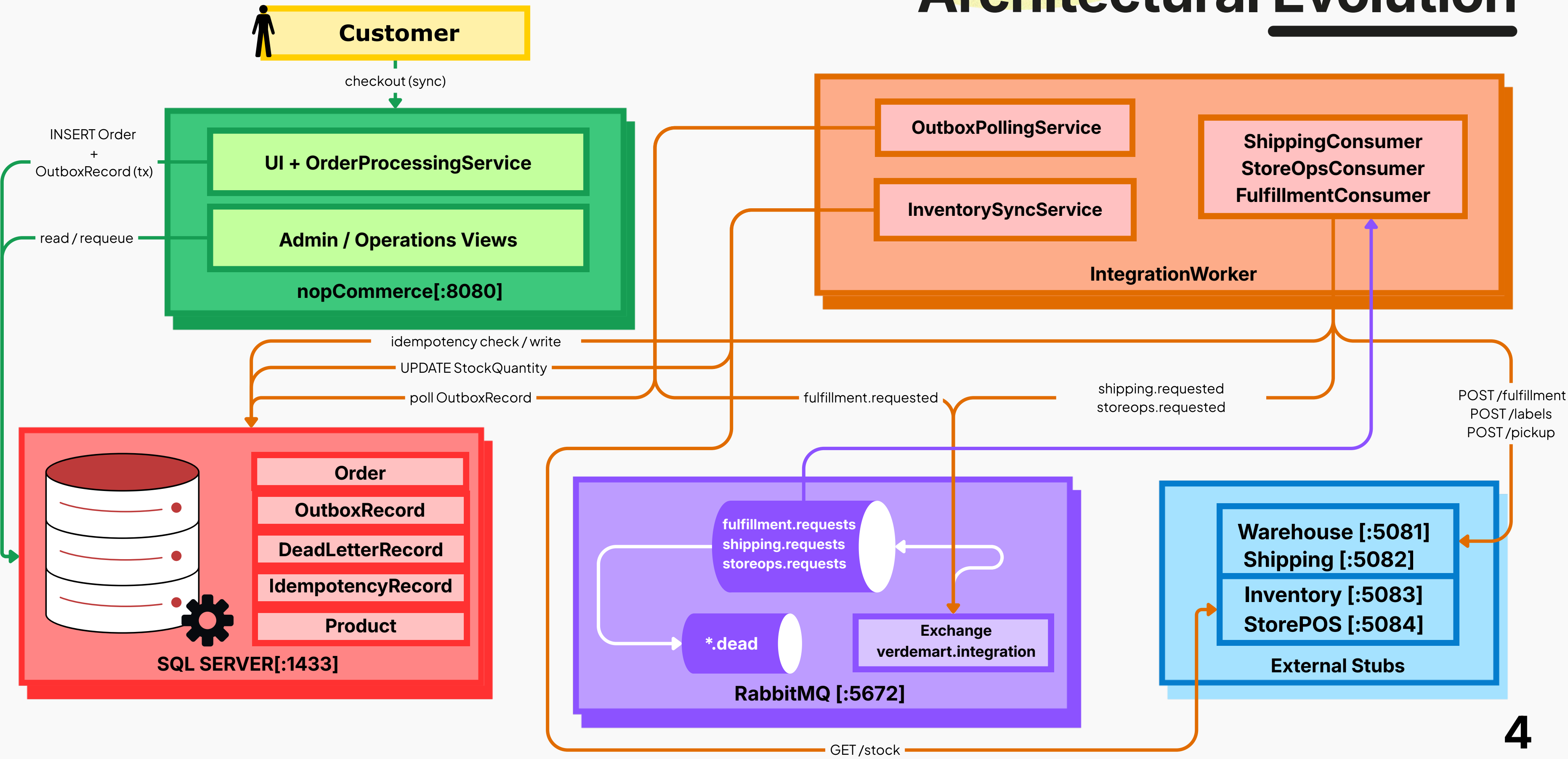
OBJECTIVE

Evolve nopCommerce from a web store into an **omnichannel commerce core** that connects online sales with warehouse management, point of sale, shipping providers, and customer support.

MAIN FOCUS

Add **reliable asynchronous integration for orders, stock, and shipping using the Outbox pattern, idempotency, and dead-letter queues** so nopCommerce handles slow or unavailable external systems without losing orders. Failed deliveries automatically raise customer support tickets.





QUALITY ATTRIBUTES

Architectural Drivers

Architectural Drivers	Key Metrics
Availability Ensure orders are never lost even if external systems fail	• 0 orders lost • Automatic recovery < 5 min • Pending visible < 30s
Consistency Allow temporary inconsistencies, but make them visible and recoverable	• 100% inconsistencies visible • Reconciliation time < 1 min • Overselling < 1%
Performance Maintain fast user experience during peaks and backend sync	• p95 checkout < 2s • Order-read < 1s • 99% events published < 60s
Modifiability Enable new channels without changing core systems	• Changes in ≤ 1 adapter + 1 boundary • Zero DB coupling • No regression in existing tests
Recoverability Retain failed integration work and resume automatically after dependency recovery	• 100% failed requests retained in retry queue • Operator visibility of outage state < 30s • 95% labels created < 10 min after recovery

QUALITY ATTRIBUTES

QA SCENARIOS SUMMARY

Scenario	Driver	Architectural Response
Warehouse Failure	Availability + Recoverability	Accept order locally and retry fulfillment asynchronously.
Stale Inventory	Consistency	Mark stale stock, revalidate checkout, and reconcile conflicts.
Order Burst	Performance	Keep checkout responsive while backend synchronization runs asynchronously.
Shipping Outage	Recoverability + Availability	Store failed label requests and resume processing after provider recovery.
New Operational Channel	Modifiability	Add new systems through adapters without rewriting checkout or sharing the database.
Inventory Conflict	Consistency	Detect cross-channel stock discrepancy, flag conflict, bound checkout to the lower reported quantity, and queue a reconciliation task for operators.

Final ADRs

#	Decision	Driving Force	Damage accepted
ADR-1	Async Fulfillment Propagation	Checkout must not depend on warehouse availability	Fulfillment state is eventually consistent
ADR-2	Transactional Outbox	Order and integration task must be atomic	Polling interval introduces propagation delay
ADR-3	Adapter Pattern, No Shared DB	Each boundary is independently replaceable	Every new system requires a dedicated adapter
ADR-4	Monolith Retained as Commerce Core	No distributed transactions without QAS justification	Commerce core cannot scale independently
ADR-5	Idempotency Keys per Message	Duplicate fulfillment or shipping labels cause operational harm	48h deduplication window; adapters must implement checks
ADR-6	Exponential Backoff + Circuit Breaker	Recovering adapter must not be overwhelmed	~50 min max delay before dead-letter
ADR-7	Dead-Letter Queue + Operator Model	No paid order is silently abandoned	Human step required in permanent failure recovery

Final ADRs

ADR-1 & ADR-2 (Most Critical)

ADR-1 — Asynchronous Fulfillment Propagation

Why

Warehouse timeout must not block checkout (QAS-1)

Decision

Integration task written outside the customer request path

Rejected

Synchronous call — checkout depends on warehouse uptime

Dominant Forces

Fault isolation · Data consistency · Supportability

Accepted Damage

Fulfillment eventually consistent; intermediate order states required; idempotency mandatory (→ ADR-5)

ADR-2 — Transactional Outbox

Why

Async alone loses tasks on crash between commit and publish

Decision

Outbox record written in same DB transaction as the order

Rejected

Publish-after-commit — silent loss on process restart

Dominant Forces

Data consistency · Fault isolation · Change cost

Accepted Damage

Polling delay between order confirmation and warehouse notification; at-least-once delivery requires all consumers to be idempotent

DEGRADATION & RECOVERY

NORMAL FLOW

Order placed → Outbox → Warehouse notified

DEGRADED FLOW

Warehouse down → Checkout still succeeds →
Task pending → Retry w/ backoff

RECOVERY FLOW

Warehouse back up → Outbox processes pending
tasks → Fulfillment sent → 0 orders lost

OBSERVABLE BEHAVIOR



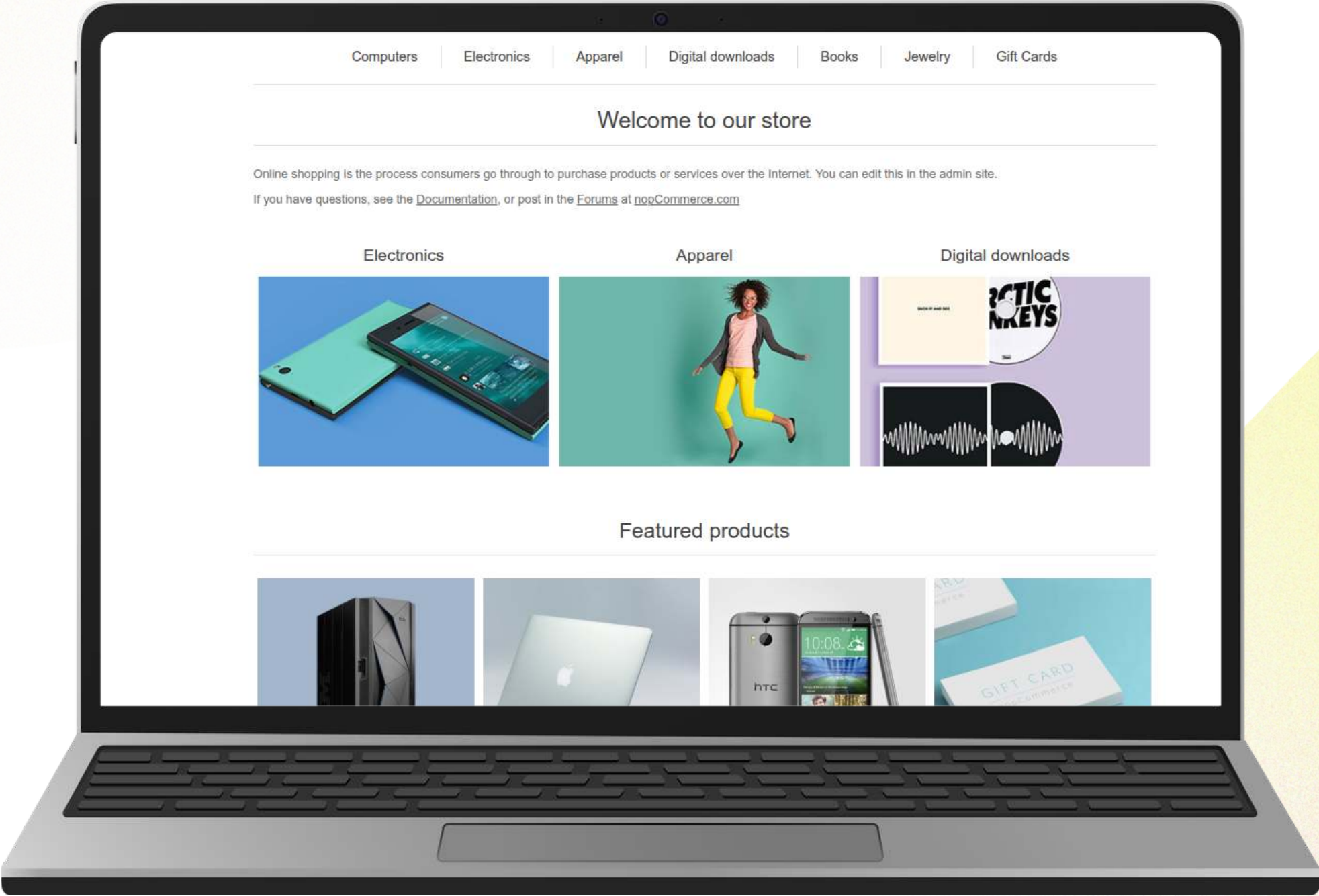
Operations View shows pending/retrying tasks in real time
Circuit breaker opens after 5 consecutive failures
Dead-letter raised if retries exhausted

RECOVERY PATH



Automatic – no manual intervention for transient failures
Operator requeue available for dead-letter entries

LIVE DEMO



TRADE OFFS

1. Availability vs Consistency

By accepting orders into the outbox before contacting the warehouse, the system is always available for checkout, but the order state is temporarily inconsistent. The customer sees "order confirmed" while the warehouse may not know about it yet. This is an explicit choice: we prioritised not losing sales over immediate consistency.

2. Deployment Simplicity vs Fault Isolation

The Integration Worker is a single process with all adapters. Simple to deploy and debug, but if the worker crashes, all adapters stop simultaneously. A more resilient design would isolate each adapter into its own process or container, but that adds operational complexity that wasn't justified for this scope.

3. Async Recovery vs Operator Burden

The retry and circuit breaker system handles most failures automatically. But when retries are exhausted, a human has to click Requeue in the Operations View. This is intentional, fully automatic requeue could cause infinite loops on messages with bad payloads. The tradeoff is safety over automation.

TRADE OFFS

4. Monolith Retained vs Integration Overhead

We kept nopCommerce as a monolith (ADR 4) instead of extracting services. This means checkout is fast and transactional, but every new integration requirement means a new adapter talking to the same monolith, which will eventually become a bottleneck as the number of channels grows.

5. Polling vs Event-Driven Inventory

The inventory sync is a polling loop (every 30s), not event-driven. This means there is always a window where stock is stale, you cannot reduce it below the poll interval without increasing DB and network load. A push-based design (WMS publishing events) would eliminate this window but requires the external system to support it.

LIMITATIONS

1. Circuit Breaker State Lost on Restart

The circuit breaker state lives in memory in the worker. If the worker restarts, all circuits reset to Closed, regardless of whether the adapter was actually healthy. A fresh worker will immediately start sending requests to a still-degraded adapter, causing a new burst of failures before the circuit opens again.

2. No Dead-Letter Alerting

When a fulfillment or shipping request exhausts all retries and lands in the dead-letter queue, no external alert fires: no email, or webhook. The operator must actively check the Operations View to discover failures. In production, silent dead-letter accumulation is an operational risk.

3. Integration Worker shares nopCommerce SQL instance

Worker reads/writes OutboxRecord, DeadLetterRecord, InventoryProjectionRecord in the same SQL Server as the web app. Not a boundary violation (these are integration-owned tables), but in full production they could move to a dedicated DB for deployment independence.

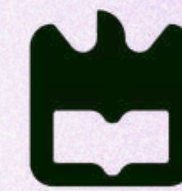
THANK YOU nopCommerce

ARCHITECTURAL EVOLUTION FINAL DELIVERABLE

QUESTIONS?

Presented by:

- Tomás Brás N° 112665
- Hugo Ribeiro N°113402
- Afonso Ferreira N°113480
- Rodrigo Abreu N°113626



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática



Computers

Electronics

Apparel

Digital downloads

Books

Jewelry

Gift Cards

Welcome to our store

Online shopping is the process consumers go through to purchase products or services over the Internet. You can edit this in the admin site.

If you have questions, see the [Documentation](#), or post in the [Forums](#) at [nopCommerce.com](#)

Electronics



Apparel



Digital downloads



Featured products

